Lab 02 — CIA Triad and AAA on Linux

> **SY0-701 Objective 1.2** — Summarize fundamental security concepts.
>
> **Domain 1** — General Security Concepts (12%) · **Time:** ~35 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Enforce confidentiality, integrity and availability on one host, then walk through authentication of a person and of a system, least-privilege authorization, and the accounting trail that proves who did what.

You'll build: An owner-only secret file, a SHA-256 integrity baseline, a self-healing systemd service, a verified password hash, an SSH key login for a service identity, a one-command sudo rule, and an accounting report from the logs.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 01 is helpful but not required; tools installed in step 1.

Background

- **Confidentiality** means only authorised subjects can read data, **integrity** means data is not altered without authorisation, and **availability** means systems and data are reachable when needed.
- **Non-repudiation** is a related property: a subject cannot credibly deny an action, because a digital signature or a trustworthy log ties the action to them.
- **AAA** separates three questions: **authentication** (who are you?), **authorization** (what may you do?) and **accounting** (what did you do?).
- We authenticate **people** with factors (something you know, have or are) and **systems** with keys, certificates and machine identities.
- Authorization models include DAC (owner-set Linux permissions), MAC (system-enforced labels such as SELinux), RBAC (roles and groups) and ABAC (attributes and context). All should follow **least privilege**.

Steps

1. Install the tools and create the lab users

`alice` is a payroll analyst, `bob` is another employee, and `svc-backup` is a non-human service identity that will log in as alice with a key.

```
apt-get update -qq && apt-get install -y openssh-server sudo curl python3
mkdir -p /run/sshd && systemctl enable --now ssh
id alice >/dev/null 2>&1 || useradd -m -s /bin/bash alice
id bob   >/dev/null 2>&1 || useradd -m -s /bin/bash bob
```

2. Confidentiality: restrict a file to its owner

`chmod 600` gives read and write to the owner only. This is discretionary access control (DAC): the owner sets the permissions.

```
echo "salary: 120000" > /home/alice/secret.txt
chown alice:alice /home/alice/secret.txt
chmod 600 /home/alice/secret.txt
ls -l /home/alice/secret.txt
sudo -u bob cat /home/alice/secret.txt 2>&1 || echo "DENIED - confidentiality enforced"
```

```
-rw------- 1 alice alice 15 ... /home/alice/secret.txt
cat: /home/alice/secret.txt: Permission denied
DENIED - confidentiality enforced
```

3. Integrity: baseline a hash and detect tampering

A SHA-256 digest is a fingerprint of the file. Any change, however small, produces a different digest.

```
sha256sum /home/alice/secret.txt > /root/secret.sha256
sha256sum -c /root/secret.sha256
echo "salary: 990000" > /home/alice/secret.txt
sha256sum -c /root/secret.sha256 || echo "INTEGRITY FAILED - file was altered"
```

4. Availability: make a service self-heal

Create a small web service with `Restart=always`, kill it, and watch systemd bring it back. Redundancy and automatic recovery are availability controls.

```
mkdir -p /srv/cia-web && echo "service is up" > /srv/cia-web/index.html
cat > /etc/systemd/system/cia-web.service <<'EOF'
[Unit]
Description=CIA lab web service (availability demo)
[Service]
ExecStart=/usr/bin/python3 -m http.server 8080 --directory /srv/cia-web
Restart=always
RestartSec=2
[Install]
WantedBy=multi-user.target
EOF
systemctl daemon-reload && systemctl enable --now cia-web
sleep 2 && curl -s http://127.0.0.1:8080/
systemctl kill -s SIGKILL cia-web
sleep 4 && curl -s http://127.0.0.1:8080/
systemctl show cia-web -p NRestarts
```

```
service is up
service is up
NRestarts=1
```

5. Authentication of people: verify a password hash

Set alice's password (something you know) and store it as a salted SHA-512 hash. Then do what PAM does at login: recompute the hash with the stored salt and compare.

```
echo 'alice:Lab1234!' | chpasswd -c SHA512
STORED=$(getent shadow alice | cut -d: -f2)
SALT=$(echo "$STORED" | cut -d'$' -f3)
echo "stored hash: $STORED"
[ "$(openssl passwd -6 -salt "$SALT" 'Lab1234!')" = "$STORED" ] && echo "correct password:
AUTHENTICATED"
[ "$(openssl passwd -6 -salt "$SALT" 'Guess123')" = "$STORED" ] || echo "wrong password:
REJECTED"
```

6. Authentication of systems: log in with an SSH key

Automation should authenticate with a key pair, not a password. The server also authenticates itself to the client with its host key.

```
[ -f /root/svc_backup_key ] || ssh-keygen -t ed25519 -N '' -C 'svc-backup@lab' -f
/root/svc_backup_key -q
install -d -m 700 -o alice -g alice /home/alice/.ssh
cat /root/svc_backup_key.pub >> /home/alice/.ssh/authorized_keys
chown alice:alice /home/alice/.ssh/authorized_keys && chmod 600
/home/alice/.ssh/authorized_keys
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
ssh -i /root/svc_backup_key -o BatchMode=yes -o StrictHostKeyChecking=accept-new
alice@127.0.0.1 'echo "logged in as $(whoami) with a key"'
```

7. Authorization: grant one least-privilege sudo command

alice is authenticated, but she should be authorised only to restart the web service. Use a validated drop-in file rather than editing `/etc/sudoers` directly.

```
cat > /etc/sudoers.d/alice-ops <<'EOF'
alice ALL=(root) NOPASSWD: /usr/bin/systemctl restart cia-web
EOF
chmod 440 /etc/sudoers.d/alice-ops && visudo -cf /etc/sudoers.d/alice-ops
sudo -l -U alice | tail -2
sudo -u alice sudo -n /usr/bin/systemctl restart cia-web && echo "ALLOWED - restart cia-
web"
sudo -u alice sudo -n /usr/bin/cat /etc/shadow 2>&1 | tail -1 || true
```

```
/etc/sudoers.d/alice-ops: parsed OK
ALLOWED - restart cia-web
sudo: a password is required
```

8. Accounting: trace who did what in the logs

Every sudo decision and SSH login is recorded with a timestamp and user. This trail supports investigations and non-repudiation.

```
journalctl _COMM=sudo --no-pager | grep -E "alice" | tail -4
journalctl -u ssh --no-pager | grep -E "Accepted publickey for alice" | tail -2
grep -hE "sudo:|sshd.*Accepted" /var/log/auth.log 2>/dev/null | tail -4 || true
```

Test it

```
sudo -u bob cat /home/alice/secret.txt 2>/dev/null || echo "C: bob denied"
sha256sum -c /root/secret.sha256 2>/dev/null || echo "I: tampering detected"
curl -s http://127.0.0.1:8080/ && systemctl show cia-web -p NRestarts
sudo -l -U alice | grep -c "systemctl restart cia-web"
journalctl _COMM=sudo --no-pager | grep -c "USER=root ; COMMAND=/usr/bin/systemctl restart
cia-web"
```

Expected: `C: bob denied`, `I: tampering detected`, `service is up` with `NRestarts=1` or higher, a sudo rule count of `1`, and at least one sudo log entry for alice's allowed restart.

Troubleshooting

Symptom	Fix
`ssh: connect to host 127.0.0.1 port 22: Connection refused`	Start the daemon with `systemctl start ssh` (on Ubuntu 24.04 also try `systemctl start ssh.socket`) and confirm with `ss -tlnp | grep :22`.
`chpasswd: unrecognized option '-c'` on an older image	Use `echo 'alice:Lab1234!' | chpasswd --crypt-method SHA512`; both spellings set the SHA-512 scheme.
`curl` returns nothing straight after the kill	`RestartSec=2` delays the restart; wait a few seconds and run `systemctl status cia-web --no-pager`.

Challenge

Add **non-repudiation** to the accounting trail: have alice sign a short statement with an SSH key (`ssh-keygen -Y sign -f <key> -n file statement.txt`) and verify it with `ssh-keygen -Y verify` using an allowed-signers file. Explain why a log entry alone is weaker evidence than a signature.

Reflection

alice successfully logged in but could not read `/etc/shadow`. Which AAA function stopped her, which one let her in, and which one proves both events happened?

Exam tips

- Hashing gives **integrity** only; a digital signature adds authenticity and **non-repudiation**.
- Authentication proves identity, authorization decides permissions, accounting records actions. sudo touches all three.
- Systems authenticate with **certificates, keys and service accounts**; people authenticate with knowledge, possession and biometric factors.
- DAC = owner decides (Linux permissions); MAC = labels enforced by the system (SELinux); RBAC = roles; ABAC = attributes and context.

References

- NIST SP 800-63B Digital Identity Guidelines (authentication) — https://pages.nist.gov/800-63-3/sp800-63b.html
- NIST SP 800-207 Zero Trust Architecture — https://csrc.nist.gov/pubs/sp/800/207/final
- sudoers manual — https://www.sudo.ws/docs/man/sudoers.man/

- systemd.service `Restart=` documentation — https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html

---